FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos II

Unidad 5 / Semana 11

Año: 2025

Contenidos

Sistemas operativos centralizados y distribuidos. Arquitecturas de Sistemas Distribuidos: Modelos de comunicación cliente-servidor, peer-to-peer y multicapas.

Índice

Introducción **3**

1. Sistemas distribuidos **4**

1.1 Tipos de Sistemas Distribuidos 5

1.2 Arquitecturas Distribuidas 5

1.2.1 Arquitectura cliente-servidor 6

1.2.2 Modelo Peer-to-Peer (P2P) 9

1.2.3 Arquitecturas Multi-Nivel 12

Introducción

Es muy importante comprender la evolución desde los modelos de sistema operativos centralizados hacia los distribuidos, especialmente en contextos donde la escalabilidad, la colaboración y la eficiencia son importantes.

Los sistemas operativos centralizados se caracterizan por operar sobre una única máquina, donde todos los recursos —CPU, memoria, almacenamiento— están concentrados y gestionados localmente. Si bien pueden tener terminales remotas conectadas, estas no participan activamente en el procesamiento, y el trabajo colaborativo entre usuarios o procesos es prácticamente inexistente.

Este enfoque, típico de sistemas de tiempo compartido (time sharing), presenta serias limitaciones cuando se busca escalar o distribuir la carga de trabajo.

En contraste, los sistemas operativos distribuidos surgen como respuesta a estas limitaciones, permitiendo que múltiples nodos conectados trabajen de forma coordinada hacia un objetivo común.

En estos sistemas, los recursos están repartidos entre distintas regiones o canales, y cada componente puede colaborar activamente en la ejecución de tareas. Esta arquitectura no solo mejora la escalabilidad, sino que también ofrece mayor tolerancia a fallos y flexibilidad operativa.

Mientras los sistemas centralizados son difíciles de escalar y carecen de colaboración efectiva, los distribuidos promueven la interconexión, el trabajo conjunto y la distribución inteligente de recursos.

Este modelo conceptual representa cómo operan hoy en día plataformas como Kubernetes, Hadoop o los sistemas en la nube, donde la eficiencia depende de la capacidad de distribuir procesos y datos entre múltiples nodos.

Sist. Centralizados	Sist. Distribuidos
Sin conexiones Con conexiones pero: • Sin trabajo colaborativo • Sin un objetivo común • Sistemas de tiempo compartido • 'Terminales de conexión' Muy difíciles de escalar: • CPUs, Memoria, HD	Componentes conectados y: • realizando trabajo colaborativo • buscando un objetivo común Escalan distribuyendo trabajo y recursos: • Nodos, regiones, canales

1. Sistemas distribuidos

Un **sistema distribuido** (SD) es un conjunto de componentes de hardware o software que, ubicados en diferentes computadoras conectadas en red, trabajan de manera **coordinada** para funcionar como un sistema único. Desde la perspectiva del usuario, esta división es transparente; parece que está interactuando con una sola entidad.

En un sistema distribuido, los componentes se comunican y coordinan sus acciones a través del **envío de mensajes**. La idea de un componente como una pieza funcional autónoma y con interfaces bien definidas es clave. En el ámbito informático, cada componente es una parte modular de un sistema de software más grande.

> Un **sistema distribuido** es una colección de computadoras independientes que dan al usuario la impresión de constituir un único sistema coherente.

La principal razón para crear sistemas distribuidos es el **compartir recursos**, lo que permite una mayor eficiencia y disponibilidad. Además, esta arquitectura fomenta un **diseño de software modular** donde cada componente puede ejecutarse en la plataforma que mejor se adapte a sus necesidades específicas. Esta especialización mejora el rendimiento del servicio.

Por ejemplo, un sistema distribuido puede tener un componente dedicado exclusivamente a la gestión y manipulación de datos. Este componente podría ser un **Sistema de Gestión de Bases de Datos (SGBD)**, diseñado para ofrecer un acceso a los datos que sea eficiente, correcto y consistente, aprovechando al máximo su especialización.

Objetivos

- El **principal objetivo** de un sistema distribuido es facilitar a los usuarios (y a las aplicaciones) el acceso a los recursos remotos (físicos, bases de datos, archivos, redes, etc.), y compartirlos de manera controlada y eficiente.
- **Transparencia**: Busca ocultar al usuario la naturaleza distribuida del sistema. Esto incluye:
  - **Transparencia de acceso**: Oculta cómo se accede a los recursos remotos.
  - **Transparencia de ubicación**: Los recursos se encuentran sin importar su ubicación física.
  - **Transparencia de concurrencia**: Múltiples usuarios pueden usar el mismo recurso simultáneamente sin interferencias.
  - **Transparencia de migración**: Los recursos pueden moverse sin afectar la forma en que se usan.
- **Fiabilidad**: Si un nodo falla, el sistema en su conjunto puede seguir funcionando. Los recursos y datos pueden replicarse en varios nodos, lo que aumenta la tolerancia a fallos.

- **Rendimiento**: Un SO Distribuido puede procesar tareas en paralelo utilizando múltiples CPUs, lo que mejora la velocidad y la eficiencia. El trabajo se distribuye entre las máquinas disponibles.

1.1 Tipos de Sistemas Distribuidos

Los sistemas distribuidos se pueden clasificar según su arquitectura subyacente:

- **Sistemas de Redes de Computadoras**: Cada máquina tiene su propio sistema operativo local, y la red permite la comunicación entre ellas. Los usuarios inician sesión en máquinas específicas y utilizan la red para acceder a servicios remotos (por ejemplo, acceder a un archivo en un servidor de red).
- **Sistemas Multiprocesadores**: Múltiples procesadores comparten la misma memoria principal y operan bajo un solo sistema operativo. La comunicación entre los procesadores es muy rápida, ya que no depende de la red. Son ideales para tareas que requieren un alto grado de paralelismo.

Cuadro comparativo de ventajas y desventajas de los sistemas distribuidos:

Ventajas	Desventajas
Mayor fiabilidad: La falla de un componente no detiene todo el sistema.	Complejidad: Diseñar y gestionar un SOD es más complejo.
Mejor rendimiento: El procesamiento se distribuye en varias máquinas, acelerando las tareas.	Seguridad: La red introduce múltiples puntos de vulnerabilidad.
Escalabilidad: Es fácil añadir más nodos o computadoras al sistema para aumentar la capacidad.	Costo: Requiere más hardware y software de comunicación.
Compartir recursos: Los usuarios pueden acceder a hardware y software de forma eficiente.	Latencia: La comunicación entre nodos a través de la red puede ser lenta.
Comunicación: Facilita la comunicación entre usuarios mediante la red.	Gestión de software: La instalación y actualización de software puede ser un desafío.

1.2 Arquitecturas Distribuidas

Las **Arquitecturas Distribuidas** (AD) son un conjunto de principios, políticas y directrices que guían la construcción de sistemas distribuidos. Su principal objetivo es establecer un marco de trabajo coherente para diseñar y desarrollar estos sistemas, buscando tres metas clave:

- **Simplificar el diseño:** Proporcionan una estructura unificada que facilita la creación y el mantenimiento del sistema. Esto estandariza el proceso de desarrollo, haciendo que el trabajo sea más eficiente y predecible.
- **Optimizar los costos:** Al promover un diseño modular y estandarizado, las AD permiten una gestión más eficiente de los recursos de desarrollo, lo que contribuye a reducir los gastos generales.

- **Fomentar la reutilización:** Ayudan a crear componentes bien definidos y autónomos que pueden ser reutilizados en distintas partes del sistema o en otros proyectos. Esto acelera el desarrollo y mejora la consistencia.



Lectura complementaria: Principales ejemplos de sistemas distribuidos: comprensión de su funcionalidad | Hivenet

1.2.1 Arquitectura cliente-servidor

La **arquitectura cliente-servidor** es un modelo de diseño para sistemas distribuidos donde las tareas están divididas entre dos tipos de componentes principales: **clientes** y **servidores**.

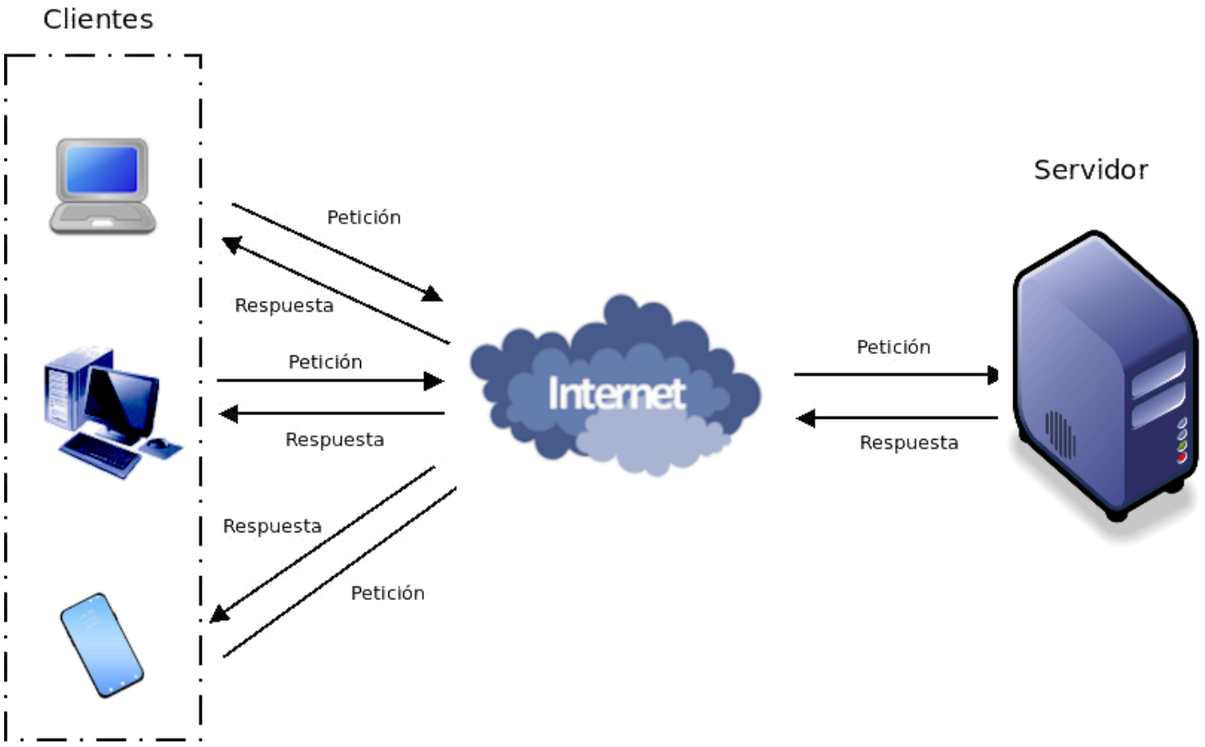
El **cliente** es el componente que solicita un servicio, y el **servidor** es el que lo provee.

Clientes y servidores son software (procesos) que pueden correr en distintos dispositivos.

Conceptos importantes:

Conceptos clave:

- **Cliente:** Un programa o dispositivo que inicia una solicitud de servicio al servidor.
- **Servidor:** Un programa o sistema que escucha las peticiones del cliente y le proporciona los recursos o servicios solicitados.
- **Petición/Respuesta:** El cliente envía una petición y el servidor responde con los datos o servicios solicitados.
- **Redes:** Las comunicaciones entre cliente y servidor siempre ocurren a través de redes de computadores.
- **Protocolos:** Se utilizan para la comunicación, como los protocolos de red.
- **Inspector:** Una herramienta que permite observar, analizar y depurar el flujo de datos entre el cliente y el servidor.



Modelo cliente-servidor

1. **El cliente inicia la comunicación.** Un cliente puede ser una aplicación, como un navegador web, que necesita acceder a datos o ejecutar una función.
2. **Envía una solicitud al servidor.** La solicitud viaja a través de una red.
3. **El servidor procesa la solicitud.** El servidor recibe la petición, la procesa y genera una respuesta.
4. **El servidor envía la respuesta al cliente.** La respuesta viaja de vuelta al cliente.
5. **El cliente recibe y presenta la información.** El cliente recibe la respuesta del servidor y la presenta al usuario de una forma comprensible (por ejemplo, mostrando una página web).

Características del modelo

- **Separación de responsabilidades:** El cliente se encarga de la interfaz de usuario y de las solicitudes, mientras que el servidor gestiona los datos, la lógica de negocio y los recursos compartidos. Esto simplifica el diseño y el mantenimiento.
- **Centralización de datos:** Los datos se almacenan y gestionan en el servidor, lo que facilita el control de la seguridad, la consistencia y las copias de seguridad.

- **Escalabilidad:** Se pueden añadir más servidores para manejar un mayor número de solicitudes de clientes, permitiendo que el sistema crezca.
- **Interoperabilidad:** Clientes y servidores pueden estar implementados en diferentes lenguajes de programación y sistemas operativos, siempre y cuando se adhieran a un protocolo de comunicación común (como HTTP para la web).

Un ejemplo común de esta arquitectura es cuando usas tu navegador (**cliente**) para visitar un sitio web. El navegador le solicita la página al servidor web del sitio, que a su vez la envía de regreso para que la puedas ver.

El cliente y el servidor: roles y funciones

Se denomina **cliente** al proceso que inicia el diálogo o solicita los recursos y **servidor**, al proceso que responde a las solicitudes.

Cliente

- Es un programa de aplicación que ejecutan los usuarios cuando necesitan acceso a recursos remotos.
- Es interactivo, dado que se ejecuta en primer plano.
- Corre de forma local en la computadora del usuario.
- Es activo dado que inicia las conexiones con un proceso servidor.
- Es un proceso temporal, dado que su vida se limita a las necesidades de interacción del cliente
- Un pedido (request) indica qué recurso se desea

Servidor

- Es un programa de aplicación que provee un servicio específico.
- No es interactivo, dado que generalmente corre en segundo plano.
- Generalmente se lo activa cuando el sistema se inicializa (demonio/servicio).
- Corre en una computadora con recursos compartidos entre varios procesos del mismo tipo.
- Espera de forma pasiva por pedidos de conexiones (escucha).
- Recibido un request, resuelve cómo satisfacerlo.

Aplicación de capas

A través del tiempo, el modelo cliente-servidor ha estado sujeto a muchos debates y controversias. Una de las principales cuestiones fue cómo establecer una diferencia clara entre un cliente y un servidor. No sorprende que con frecuencia ésta no exista. Por ejemplo, un servidor para una base de datos distribuida puede actuar continuamente como un cliente, ya que reenvía solicitudes a diferentes servidores de archivos responsables de implementar las tablas de la base de

datos. En tal caso, el propio servidor de la base de datos no hace más que procesar consultas.[1]

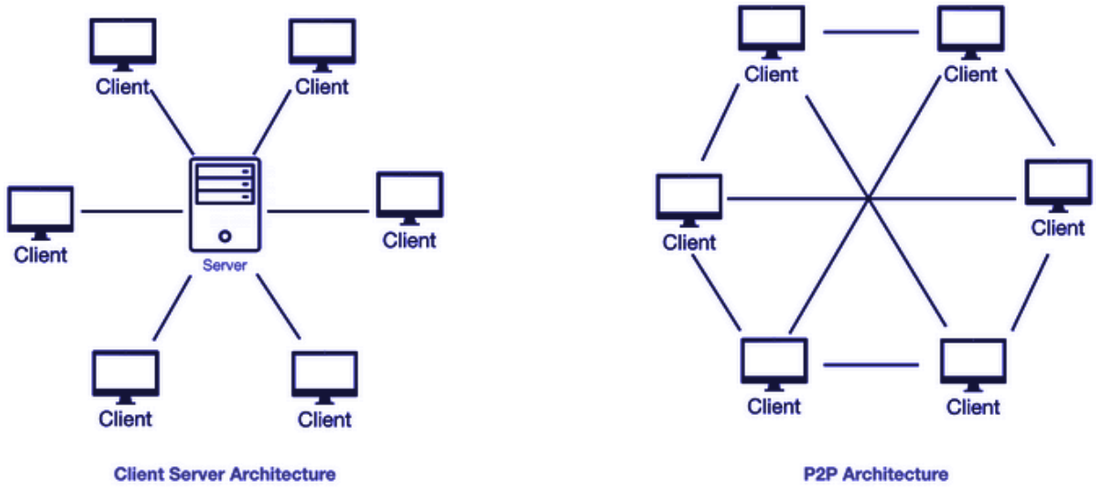
Sin embargo, si consideramos que muchas aplicaciones cliente-servidor están enfocadas en dar a los usuarios acceso a las bases de datos, mucha gente ha defendido una diferencia entre los siguientes tres niveles, siguiendo básicamente el estilo arquitectónico en capas:

1. El nivel de interfaz de usuario.
2. El nivel de procesamiento.
3. El nivel de datos.

Luego profundizaremos en el tema de las capas en los Arquitecturas Multi-Nivel.

1.2.2 Modelo Peer-to-Peer (P2P)

Un sistema **Peer-to-Peer (P2P)** es un **Sistema Distribuido (SD)** en el cual todos los elementos interconectados desempeñan un **papel idéntico**. Esto contrasta fundamentalmente con el tradicional **modelo cliente-servidor**.



Esquema comparativo de la arquitectura cliente-servidor y P2P

Objetivos y fundamentos del modelo P2P

El modelo P2P busca principalmente **posibilitar el compartir datos y recursos a muy gran escala**. Una de sus mayores ventajas es que **no requiere de servidores dedicados ni de su infraestructura** asociada. En su lugar, el sistema proporciona

[1] Tanenbaum Andrew S.y Van Steen Maarten (2008): Sistemas Distribuidos. Principios y Paradigmas Segunda edición.

servicios y aplicaciones distribuidas utilizando los datos y **recursos computacionales disponibles** en ordenadores personales y estaciones de trabajo conectados a Internet.

Descentralización y Búsqueda

Dado que la información **no se almacena en unos cuantos servidores centralizados**, su ubicación y posterior recuperación se convierte en un aspecto esencial y **no trivial** en el diseño de sistemas P2P.

Este diseño se caracteriza por:

- Un **servicio totalmente descentralizado y auto-organizado**, aunque esto se logra "con matices" que pueden variar según la arquitectura específica.
- Un **equilibrio dinámico de las cargas** de almacenamiento y procesamiento entre todos los nodos (computadoras) que participan en el sistema.

Características esenciales de un sistema P2P

Un sistema P2P se define por una serie de características inherentes a su arquitectura descentralizada:

- **Contribución Universal:** Todo usuario (o nodo) debe **contribuir con recursos** (almacenamiento, ancho de banda, capacidad de procesamiento) al sistema.
- **Igualdad Funcional:** Todos los nodos comparten las **mismas capacidades y responsabilidades funcionales**.
- **Autonomía y Resiliencia:** Su correcto funcionamiento **no depende** de la existencia de ningún sistema administrado de forma centralizada.
- **Anonimato (Opcional):** El sistema puede ser diseñado para ofrecer un **grado limitado de anonimato** tanto a los proveedores como a los usuarios de los recursos.
- **Eficiencia Algorítmica:** Su funcionamiento eficiente se basa críticamente en el **algoritmo implementado para la ubicación de datos** y el **acceso equilibrado** a estos.

Ventajas de redes P2P

Uno de los beneficios más significativos de las redes P2P es su **escalabilidad**. Con cada nuevo nodo añadido a la red, la capacidad del sistema para manejar el tráfico mejora, reduciendo la dependencia de un solo servidor y mejorando el rendimiento general.

En términos de **eficiencia**, las redes P2P también tienen ventaja sobre los sistemas tradicionales de cliente-servidor. Dado que la distribución del contenido ocurre directamente entre pares sin involucrar a un servidor central, el ancho de banda se utiliza de manera más eficaz. Este enfoque también reduce el costo de la infraestructura y el ancho de banda para las empresas que ofrecen servicios P2P, lo que conlleva un ahorro sustancial de costes.

Hablando de **resiliencia**, la naturaleza descentralizada de las redes P2P les permite seguir funcionando incluso cuando uno o más nodos fallan. No hay un solo punto de fracaso, lo que hace que estas redes sean altamente confiables. Además, las redes proporcionan un grado de anonimato, por lo que son atractivas para los usuarios que valoran la privacidad.

Por último, las redes P2P mejoran **la flexibilidad**. Permiten la unión de dispositivos y dejan la red a voluntad sin interrumpir el funcionamiento general. Los usuarios pueden compartir y acceder libremente a los recursos, aumentando la conveniencia y fomentando un entorno colaborativo.

Desventajas de las redes P2P

Sin embargo, las redes P2P no están sin sus inconvenientes. Una de las principales preocupaciones es **la seguridad**. La falta de una autoridad central para gestionar las políticas de seguridad puede hacer que estas redes sean vulnerables a diversas amenazas, incluyendo malware, intercambio ilegal de contenidos y violaciones de datos.

En cuanto a **la regulación** de **contenidos**, en las redes P2P, no hay control centralizado para evitar el uso compartido de material ilegal o de derechos de autor. Esta falta de regulación puede conducir a cuestiones jurídicas y éticas.

Calidad de servicio (QoS) también puede ser un reto significativo para los sistemas P2P. Dado que cada nodo funciona de forma autónoma, mantener una calidad constante o un estándar de servicio puede ser complicado. Por ejemplo, la velocidad de transferencia puede variar en función de la calidad de conexión de los pares involucrada en un proceso de intercambio de archivos.

En conclusión, si bien las redes P2P ofrecen enormes ventajas como la escalabilidad, la eficiencia, la resiliencia y la flexibilidad, no están exentas de sus desventajas. El reto radica en aprovechar los beneficios al tiempo que se atenúan eficazmente los posibles inconvenientes, especialmente en términos de seguridad y regulación. Cuando se usan juiciosamente, las redes P2P pueden revolucionar el futuro de la red de internet.

Aplicaciones y casos de uso de redes P2P

Cuando se explotan a todo su potencial, las redes P2P pueden impactar significativamente en diversos sectores y aplicaciones con sus capacidades y características únicas.

Compartir recursos en redes P2P

Las redes P2P sobresalen en **el intercambio** de **recursos**. Al dividir una carga de trabajo entre pares, la red comparte eficientemente recursos como ancho de banda y potencia de cómputo. Un ejemplo de esto se puede ver en aplicaciones de intercambio de archivos como BitTorrent, donde los archivos grandes se parten en trozos y se distribuyen entre pares para permitir descargas más eficientes.

Además, las redes P2P desempeñan un papel crucial en **las redes** de **entrega de contenidos (CDN).** El reparto de contenidos entre pares geográficamente dispersos reduce la carga en un solo servidor y proporciona un acceso más rápido a los usuarios.

En **criptomonedas** como Bitcoin, las redes P2P forman un libro mayor descentralizado que verifica y registra las transacciones sin necesidad de una aplicación de autoridad central que ha revolucionado el mundo de las finanzas.

Ejemplos de redes P2P en diferentes sectores

Más allá del intercambio de recursos, las redes P2P influyen en otros sectores:

- **Entretenimiento:** Aplicaciones como Spotify utilizan redes P2P para transmitir música a millones de usuarios simultáneamente, minimizando la latencia y mejorando la experiencia de escucha.
- **Juego:** Algunos juegos en línea multijugador utilizan redes P2P para reducir la latencia (el tiempo de retardo) y, consecuentemente, ofrecer una experiencia de juego más fluida y responsiva.
- **Salud:** Las redes P2P pueden ayudar a distribuir la carga computacional de los datos médicos complejos de procesamiento, como secuencias del genoma o grandes imágenes médicas.
- **Telecomunicaciones:** Servicios como Skype utilizan una estructura P2P para facilitar una comunicación directa y eficiente entre los usuarios.

Las redes P2P, con su amplia gama de aplicaciones, tienen el potencial de perturbar el funcionamiento tradicional de la industria, ofreciendo una mejor gestión de los recursos, una mayor experiencia de usuario y una mayor eficiencia. Sin embargo, debe mantenerse la integridad y la seguridad de estos sistemas para su aplicación satisfactoria en estos sectores.

1.2.3 Arquitecturas Multi-Nivel

Las **Arquitecturas Multi-Nivel** (también llamadas **N-Niveles** o **Multi-Tier**) en el contexto de los **Sistemas Operativos Distribuidos (SOD)** son un estilo de

arquitectura de *software* que se centra en **separar las responsabilidades lógicas de una aplicación en distintos niveles** (tiers), los cuales se implementan y **despliegan físicamente en diferentes máquinas** o servidores interconectados.

Diferencia Clave: Capas vs. Niveles

Es fundamental distinguir entre **capas** y **niveles** en este contexto:

Concepto	Definición	Relevancia en SOD
Capa (Layer)	División lógica del código. Define un conjunto de componentes con responsabilidades específicas (Ej: Presentación, Lógica de Negocio).	Es una separación interna del software que garantiza la coherencia y el mantenimiento.
Nivel (Tier)	División física o de despliegue. Representa la máquina o servidor donde se ejecuta una o más capas lógicas.	Es el pilar de la distribución. Permite la escalabilidad y la tolerancia a fallos al ubicar las cargas en máquinas separadas.

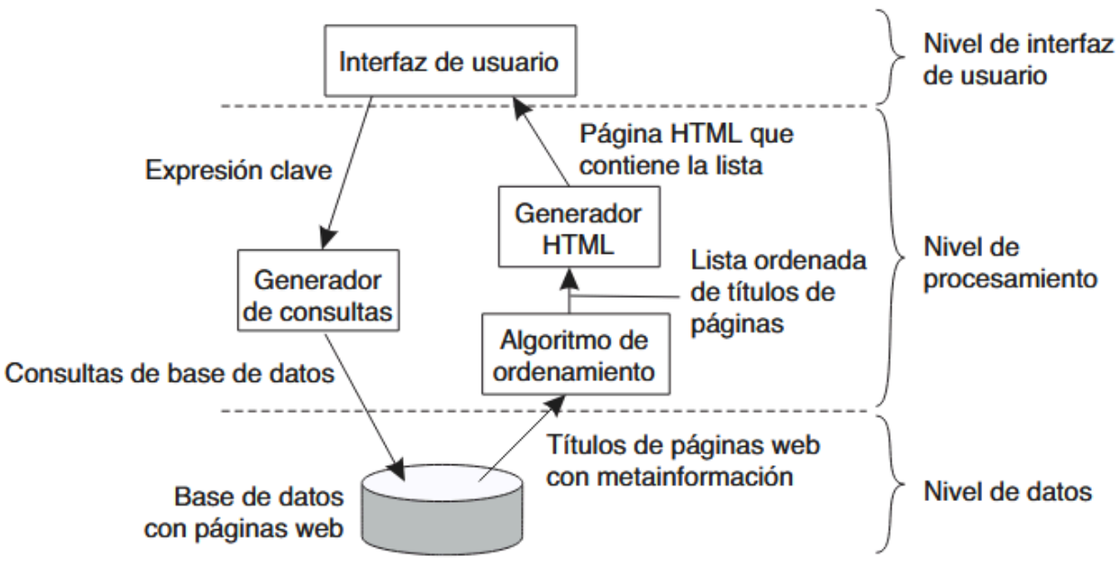
Una arquitectura multi-nivel, por lo tanto, implica que las capas lógicas están distribuidas físicamente a través de una red de computadoras.

Arquitectura de tres niveles (3-Tier)

El modelo más común y representativo de la arquitectura multi-nivel es el de Tres Niveles, especialmente aplicado a aplicaciones cliente-servidor:

1. **Nivel de Presentación (Interfaz de Usuario - IU)**
    - **Responsabilidad:** Mostrar la información al usuario y gestionar las interacciones (entradas de datos).
    - **Ubicación Típica:** Se ejecuta en el **cliente** (navegador web, aplicación de escritorio o móvil).
    - **Ejemplo:** La interfaz de una aplicación web que el usuario ve y usa.
2. **Nivel de Lógica de Negocio (Servidor de Aplicación)**
    - **Responsabilidad:** Contiene la lógica central de la aplicación. Procesa las peticiones del Nivel de Presentación, aplica las reglas de negocio y se comunica con el Nivel de Datos.
    - **Ubicación Típica:** Se ejecuta en uno o más **servidores dedicados**.
    - **Ejemplo:** Un *middleware* que calcula el precio final de un pedido, gestiona el inventario o valida una transacción.
3. **Nivel de Datos (Servidor de Base de Datos)**
    - **Responsabilidad:** Almacenamiento, gestión y acceso a los datos de la aplicación.

- **Ubicación Típica:** Se ejecuta en uno o más **servidores de bases de datos** (DBMS).
- **Ejemplo:** El servidor que aloja la base de datos relacional (MySQL, Oracle) o NoSQL (MongoDB).



Organización simplificada, en tres capas diferentes, de un motor de búsqueda en internet.[2]

Ventajas en sistemas distribuidos

La arquitectura multi-nivel es esencial para la mayoría de los sistemas distribuidos modernos por las siguientes razones:

- **Escalabilidad:** Se puede escalar cada nivel de forma **independiente**. Si el tráfico web aumenta, se añaden más servidores al Nivel de Presentación/Aplicación sin tocar el Nivel de Datos, y viceversa.
- **Reutilización:** La Lógica de Negocio (Nivel 2) puede ser utilizada por múltiples interfaces (Nivel 1), como una *web* y una *app* móvil, sin duplicar código.
- **Mantenimiento y Evolución:** Los cambios en un nivel afectan mínimamente a los demás, facilitando la actualización de tecnologías o la corrección de errores de forma modular.
- **Seguridad:** Al separar físicamente el Nivel de Datos (más sensible) del Nivel de Presentación (accesible al público), se añaden capas de seguridad y se restringe el acceso directo a la información crítica.

[2] Tanenbaum Andrew S.y Van Steen Maarten (2008): Sistemas Distribuidos. Principios y Paradigmas Segunda edición.

- **Balanceo de Carga:** La distribución en diferentes máquinas permite utilizar técnicas de **balanceo de carga** para repartir las peticiones y evitar cuellos de botella en un único servidor.



Actividad S11-A1

Realice la actividad identificada como *S11-A1* (Semana 11, Actividad 1) en la sección de actividades del campus.